

终端交互“经典坑”大全：Bug 图谱、证据链与 E2E 测试清单（Deep Research）

日期：2026-02-12

定位：交互式 CLI/REPL/TUI（Windows Console/ConPTY + POSIX PTY）

目标：做“广度聚合”——把人类在终端里被坑得很惨的典型问题，按可复现/可测试维度收敛成一份清单与测试建议。

Executive Summary

终端相关 bug 的“经典性”来自两个事实：**终端是状态机**（模式很多、兼容层很多），并且**输入/输出并非纯文本流**（包含行编辑、控制序列、编码、宽度、流控、安全副作用）。因此同一段代码在不同终端（cmd.exe vs PowerShell/Windows Terminal、xterm vs tmux、ssh/pty vs pipes）下表现差异巨大。[1][3][4]

对工程团队最有效的应对方式，是把问题分解为“可测的协议/语义单元”，用“真 TTY”（Windows: ConPTY；POSIX: pty）做 opt-in E2E 回归，并尽可能让断言锚到可观测证据链（例如交互程序自己的事件落盘/状态文件），而不是脆弱的屏幕文案。[1][4]

Key Findings

- **Console/TTY 模式位决定了按键语义**：Windows 的 `ENABLE_LINE_INPUT / ENABLE_ECHO_INPUT / ENABLE_PROCESSED_INPUT / ENABLE_VIRTUAL_TERMINAL_INPUT` 会改变应用“看到的输入是什么”。POSIX 的 `ICANON/ECHO/ISIG/IXON` 等 `termios` 标志同样会改变行编辑、信号与流控语义。[1][3]
- **粘贴（paste）是“必测”而非“锦上添花”**：Bracketed paste（`ESC[200~...ESC[201~`）是现代终端、xterm 兼容与 Readline 生态中的关键机制；它既影响功能（多行粘贴是否被逐行执行），也影响安全（粘贴内容是否触发编辑命令）。[4][5]
- **输出不是“打印字符串”这么简单**：光标移动、回车覆盖、行尾自动换行、alternate screen buffer、保存/恢复光标等控制序列，会让“视觉状态”与“内部缓冲状态”发生分离，导致“整行消失/覆盖/错位”等经典现象。[4][6]
- **Unicode 宽度/组合字符是 TUI 永恒地雷**：CJK 全角宽度（2 列）、combining marks（0 列）、以及“模棱两可宽度（Ambiguous width）”偏好会导致对齐/擦除/光标位置错乱。wcwidth 类规格与实现就是为此存在的，但不同终端偏好仍会造成差异。[7][8]

- **XON/XOFF (Ctrl+S/Ctrl+Q) 会让“终端像死机一样”**: `IXON` 开启时, Ctrl+S 会暂停输出 (看起来程序卡死), Ctrl+Q 才恢复; 这个坑非常古老且仍常见于远程/串口/某些默认配置。[3]
- **编码/代码页会把“显示问题”扩大成“交互问题”**: Windows console 有输入/输出 code page (`GetConsoleCP/GetConsoleOutputCP`)。如果程序假设 UTF-8, 中文输入就可能被解码成乱码, 从而连锁破坏“宽度计算、退格擦除、落盘断言”。[9]
- **安全维度: 终端逃逸序列注入仍被反复利用**: OSC/CSI 等控制序列可以修改标题、切换屏幕缓冲、操纵选择区/剪贴板 (如 xterm 的 selection 操作)、甚至构造“看起来像空日志”的欺骗。输出未消毒会导致日志查看器、CLI 工具链被动执行“终端副作用”。[6][10]
- **多层终端 (tmux/screen/ssh/WSL) 带来能力错配**: `TERM`、`terminfo`、`alternate screen`、`bracketed paste`、鼠标与键盘协议在多层转发时容易丢失/降级/变形, 导致“在本机 OK、进 tmux 就坏”的经典体验。[4][5]
- **“ESC 歧义”会制造编辑器/TUI 的输入延迟与错判**: 许多特殊键以 `ESC` 开头, 导致程序无法可靠区分“按了 Escape”与“按了 Alt/功能键序列”; 现实中往往只能用定时器折中 (延迟 vs 错判)。复用器 (tmux 等) 还会把这种折中放大成体感卡顿或乱码。[20][21]
- **中断/关闭语义不统一 (Ctrl+C/关闭窗口/子进程组)**: POSIX 依赖信号; Windows 依赖 console control handler + mode 位 (例如 `ENABLE_PROCESSED_INPUT` 影响 Ctrl+C 是否被当成信号)。不测的话, 长会话/流式输出时很容易出现“按了没反应 / 直接退出 / session 污染”。[1][18][19]
- **Windows ConPTY 本身也有“经典坑”(死锁/退出悬挂)**: 比如 `ClosePseudoConsole` 的 drain/线程模型要求、以及旧版本 Windows 上潜在的无限等待, 这会直接把你的 expect harness 变成 flaky 或 hang。[15][16]
- **IME (中文/日文输入法) 是 Windows 终端的长期复杂域**: 某些边界条件 (靠近右边界、组合输入、转换阶段) 会触发重复字符、挂死、崩溃等; 这类问题通常不属于你的业务代码, 但会影响你“看起来像吃字/乱序”的用户体验与可测性。[30]

Detailed Analysis

A. 输入链路 (键盘 → 终端 → 应用) 经典坑

A1) “行编辑”到底在谁那里发生?

- **Windows**: 当启用 `ENABLE_LINE_INPUT`, `ReadFile/ReadConsole` 只有在读到回车后才返回; 并且在 `ENABLE_PROCESSED_INPUT` 条件下, Backspace、回车、换行等由系统处理, 而不是按原样交给应用。[1][2]
- **POSIX**: 当启用 `ICANON`, 终端处于 canonical (行缓冲) 模式; `ECHOE` / `WERASE` / `KILL` 等机制会让“退格/删词/清行”由行规程处理。[3]

典型症状:

- “按键不灵/删不动/一下删一大片（删词）”
- “有时整行没提交/被合并成一坨（typeahead/paste coalesce）”
- “Ctrl+C 在某些模式下变成普通字符/或者直接 kill 掉程序”

可测试化建议：

- 真 TTY 下用输入序列（BS/DEL、Ctrl+S/Ctrl+Q、Ctrl+C、箭头键）驱动交互，断言应用最终接收到的“prompt 文本”（或落盘事件）符合预期，而不是依赖屏幕显示。

A2) XON/XOFF：Ctrl+S “冻结终端”

`IXON` 打开时，终端会启用 XON/XOFF 软件流控；Ctrl+S（STOP）会暂停输出，Ctrl+Q（START）恢复。[3]

典型症状：

- “程序突然卡死，键盘还能打但没反应”（其实输出被暂停）

E2E 建议：

- 用一个会持续输出的子命令（或 streaming 模式）触发输出，然后注入 Ctrl+S，观察输出停止，再 Ctrl+Q 恢复；对 `openagentic_cli` 可作为 stress 用例（opt-in）。

B. 粘贴（Paste）与多行输入：功能与安全交叉点

B1) Bracketed paste 的语义与生态地位

xterm 规范明确 bracketed paste 的开始/结束标记：`ESC[200~` 与 `ESC[201~`，用于让程序区分“粘贴”与“键入”。[4]

GNU Readline 也提供 `enable-bracketed-paste`，并指出它能防止粘贴内容触发绑定的编辑命令。[5]

典型症状：

- “粘贴多行时每行都被立即执行/触发命令”
- “粘贴里包含某些控制序列，提前终止 bracketed paste（绕过保护）”

安全扩展：

- xterm 的 FAQ/补丁记录了 bracketed paste 终止序列可被嵌入绕过，从而导致“粘贴触发执行”的安全洞，终端/壳/编辑器都踩过这个坑。[11]

E2E 建议：

- 粘贴块包含 / 开头文本、空行、以及包含 `ESC[201~` 的恶意片段（以安全方式模拟，不执行危险命令），验证程序不会误走“命令分支”。

B2) Readline 默认值变化引发的生态级回归

Readline 8.1 被指出“默认开启 bracketed paste”，导致一些程序行为变化并触发生态回归（例如 Python REPL 场景）。这说明“终端能力默认值变化”会突然让大量程序暴露出隐患。[12][13]

C. 输出链路（应用 → 终端渲染）经典坑

C1) 回车覆盖与行尾自动换行

Windows `SetConsoleMode` 文档描述了 `ENABLE_WRAP_AT_EOL_OUTPUT`、`DISABLE_NEWLINE_AUTO_RETURN` 等输出模式，这些会改变“写满一行”的行为（换行 vs 覆盖）。[2]

典型症状：

- 进度条/流式输出使用 `\r` 回车覆盖时，覆盖范围不一致，导致残影、错行、prompt 被擦。

E2E 建议：

- 在真 TTY 下模拟 streaming 输出 + 用户输入竞争（typeahead / 未回车输入），断言“功能证据链”（用户输入最终是否被读取/落盘），并把“视觉一致性”作为次级可选断言（因为终端实现差异大）。

C2) Alternate screen buffer：滚回去看不到输出

xterm 文档解释了 Normal/Alternate 两个屏幕缓冲，alternate buffer 不保留 scrollbar；很多全屏程序通过 terminfo 的 `smcup/rmcup`（或私有模式如 1049）切换。[4]

典型症状：

- 进入 TUI 后退出，之前输出“消失了”
- 某些终端/复用器禁用 ti/te 导致全屏程序渲染错乱

E2E 建议：

- 对需要全屏的 CLI/TUI 才测；对 REPL 类可只验证“不意外切到 alternate buffer”。

D. Unicode：宽度、组合字符、模棱两可宽度

`wcwidth` 的规格文档详细讨论了 combining marks 与 East Asian Width (W/F=2) 的测量规则，并指出某些字符宽度在不同终端偏好下会变化 (Ambiguous width)。[7][8]

典型症状：

- 表格对齐错位（列歪）
- 退格擦除不干净（残影）
- 光标位置与屏幕显示不一致（输入覆盖到奇怪位置）

E2E 建议：

- 用 CJK、combining、emoji（含 Regional Indicator 旗帜）、Ambiguous width 字符做对齐/擦除用例。
注意：不同终端对 Ambiguous width 的偏好不同，因此要把断言设计成“可配置或只测不歪到不可用”。

E. 编码/代码页：Windows 的“隐性炸弹”

Windows console 有独立的输入/输出 code page： `GetConsoleCP`（输入）用于把键盘输入翻译成字符， `GetConsoleOutputCP`（输出）用于把写出的字符映射到可显示 glyph。[9]

典型症状：

- 中文输入变乱码；进一步影响宽度/退格/日志
- 同一程序在 `cmd.exe`（某 code page）与 PowerShell/Windows Terminal（可能切到 UTF-8/或不同管线）下表现不同

E2E 建议：

- 明确在测试环境中固定编码（UTF-8 vs 非 UTF-8），至少覆盖“中文输入/落盘正确”。

F. 安全：终端逃逸序列注入（Terminal Injection）

xterm 文档描述了 OSC 机制，并包含 selection/clipboard 操作（例如“Manipulate Selection Data”一类控制）。这意味着**仅仅打印不可信文本**也可能在用户终端里产生副作用（改变选择区、设置窗口属性等）。[6]

安全社区与工程界反复指出：ANSI/OSC 逃逸序列仍然容易被滥用（例如日志伪装、点击链接、剪贴板注入），而责任常在“终端实现 vs 应用输出消毒”之间摇摆。[10]

E2E 建议（安全向，慎用、默认不跑）：

- 为 CLI 的“日志/回显”路径添加“输出消毒层”或“NO_COLOR/NO_OSC”开关，并用 e2e 验证：当输出包含 `ESC]52;...` / `OSC8` 等片段时，程序不会原样透传到用户终端（至少在默认安全模式下）。

G. 能力协商与多层终端 (`TERM` /`terminfo`/`tmux`/`ssh`)：“在本机 OK”失效的根源

终端世界的“能力协商”主要靠环境变量与数据库：

- `TERM` (如 `xterm-256color`、`screen-256color`、`tmux-256color`) 是应用选择“该发什么序列”的关键线索。
- `terminfo` 数据库把“某个终端的能力 → 对应控制序列/按键序列”映射起来 (例如 `backspace` 的 `kbs` / `key_backspace`、箭头键 `kcub1` 等)。[21]

进入 `tmux`/`screen`/`ssh`/`VS Code Terminal`/`WSL` 这类多层环境时，`TERM` 往往会被修改或降级，某些能力 (鼠标、`bracketed paste`、`alternate screen`) 也可能被“拦截/翻译/禁用”。结果就是：

- 同一程序在不同壳/复用器里表现不一致
- “Backspace/方向键变成奇怪字符” (终端发出的序列与 `terminfo` 期待不匹配)
- “全屏程序退出后屏幕没恢复/滚回去看不到输出” (`alternate buffer`/`ti-te/1049h/1049l` 路径被截断) [4] [20]

E2E 建议：

- 设计 E2E 时把环境分成最小覆盖集：`cmd.exe` / `PowerShell+Windows Terminal` / `WSL2+Ubuntu` / `ssh+tmux` (可选)，并显式记录每组的 `TERM` 与“是否真 TTY”。
- 至少覆盖：方向键/Backspace 不应污染输入 (即便你不支持高级编辑，也要保证“不会把控制序列当普通文本落盘”)。

H. 中断、关闭与信号：Ctrl+C 不是“一定会来”的

H1) POSIX： `ISIG` /`SIGINT`/`SIGWINCH` 这条路

在 POSIX 体系里，`Ctrl+C`、`Ctrl+Z`、窗口 `resize` (`SIGWINCH`) 等都建立在 `termios` + 信号机制上；如果程序切进 `raw` 模式却没有正确恢复，就会出现“回到 `shell` 后键盘怪异/不回显/方向键乱码”，最终用户只能 `reset` / `stty sane`。[3][23]

H2) Windows：Console Control Handlers + mode 位

Windows 的 `Ctrl+C`/`Ctrl+Break` 等更像“控制事件”：

- `SetConsoleCtrlHandler` 注册 handler；handler 在系统创建的新线程里执行，并存在一些 `shutdown` 事件的超时语义。[18]
- `ENABLE_PROCESSED_INPUT` 影响 `Ctrl+C` 是“信号”还是“普通键盘输入”。[1][18]

- `GenerateConsoleCtrlEvent` 可以向共享 console 的进程组发送 Ctrl+C/Ctrl+Break（但 Ctrl+C 不能精确限定到单个进程组）。[19]

E2E 建议：

- 把 Ctrl+C 变成 P0：分别覆盖“空闲时按 Ctrl+C”和“流式输出进行中按 Ctrl+C”；断言：不崩溃、不会把半截输入落成一条 turn、不会污染 session。

I. 键盘协议的细节坑：ESC 歧义、特殊键序列、Backspace 映射

I1) ESC 歧义与延迟（特别是 tmux/vim/emacs 生态）

tmux FAQ 对 `escape-time` 的解释很典型：特殊键序列都以 `ESC` 开头，而 Escape 键本身也是单字节 `ESC`；为区分两者，需要一个“等待后续字节”的超时，超时越大越不误判、但越卡顿；超时越小越灵敏、但越可能误判/乱码。[20]

ncurses 文档也指出：把 Escape 当单字符命令会导致“最长 1 秒的延迟”，因为它会等待判断是否是功能键序列的一部分。[22]

I2) Backspace：不仅是 BS vs DEL，还受终端协议与 terminfo 影响

在 Windows 的 VT input 模式里，微软文档明确给出：Backspace 会以 `0x7f (DEL)` 进入输入流。[14] 而在 terminfo 里，Backspace 键能力通常映射到 `kbs / key_backspace`，不同终端/配置可能选择 BS 或 DEL。[21][22]

E2E 建议：

- Backspace/DEL/BS 的兼容回归属于“经典必测”，但不要把断言绑死在屏幕表现；最好锚到事件落盘或最终发送给模型的内容。

I3) 输入法（IME）与组合输入：看起来像“吃字/重复字”

Windows Terminal 社区 issue 显示：使用 IME 在某些边界位置会出现重复字符、挂死甚至崩溃。[30] 这类问题往往不应由你的 CLI“完全修复”，但可以通过两种方式降低影响面：

- 尽量避免“屏幕级重绘 + 频繁移动光标”的复杂 repaint（把稳定性优先级放在美观之上）
- 为用户提供降级开关（例如关闭 fancy prompt/关闭 streaming 时的实时重绘）

J. 终端状态恢复：raw 模式残留、回显/光标/颜色“污染”

“终端状态没恢复”是另一类经典坑：程序退出后，用户发现 shell 不回显、光标消失、颜色全乱、换行奇怪.....

这些问题在 POSIX 上通常能用 `stty sane` 恢复（其组合设置会重置 `icanon/echo/isig/ixon` 等关键标志）。[23]

E2E 建议：

- 增加一个“退出后可继续输入”的回归：expect harness 在子程序退出后，在同一 TTY 里继续输入并读取回显/输出，确保没有残留 raw/无回显状态（这类测试非常抓住“忘了 restore mode”的 bug）。

K. 安全与“文本欺骗”：终端注入 + 双向控制字符

K1) 终端逃逸序列注入的现实性

除了“直接把 OSC/CSI 发到用户终端”的传统风险，近年也出现了大量“把不可信输入写入日志/诊断输出 → 终端被动执行控制序列”的真实漏洞通告（ANSI escape injection / log poisoning）。例如 Rust 生态的 `tracing-subscriber` 与 Ruby on Rails 的日志路径都出现过相关 CVE/通告，核心点都是：**不可信输入里夹带 ANSI 控制序列会在终端侧产生副作用**。[28][29]

K2) Trojan Source（Bidi 控制字符）让“源码/输出看起来不是它实际的样子”

Trojan Source 论文与其对应 CVE（CVE-2021-42574）讨论了双向控制字符（Bidi overrides）如何让读者/审计者在视觉上看到与真实字节序列不同的代码/文本。[26][27]

这不仅影响源码安全，也会影响“CLI 输出/日志审计”（例如让一行看起来被注释/被截断）。

E2E 建议（安全向，默认不跑）：

- 构造包含 ANSI 控制序列、Bidi 控制字符的“模型输出/工具输出”样本，验证 CLI 默认不会把它们以“可执行副作用”的方式直接透传（至少要提供安全开关/过滤策略）。

Areas of Consensus

- 不同终端 + 不同模式位 + 不同编码/宽度偏好，使得“在我机器上 OK”在终端世界里不成立；必须靠真 TTY E2E 回归兜底。[1][3][4]
- bracketed paste、alternate buffer、宽字符、流控、代码页是最常见且影响最大的坑，应该优先测试化。[3][4][7][9]

Areas of Debate

- **做多强的 line editor**：完全实现 readline 级别编辑很贵；但只靠系统行编辑又会遇到跨终端语义漂移（尤其 Windows）。[1][3]

- **Unicode 宽度的“正确性”**：按 code point vs grapheme cluster（用户感知字符）回删/对齐；以及 Ambiguous width 的取舍。[7][8]
- **安全消毒的边界**：严格过滤控制序列会破坏合法彩色输出/链接；不过不做过滤会留下终端注入面。[10]

建议的“终端经典坑”E2E Backlog（面向 openagentic_cli）

说明：这是“广度清单”，不要求一次做完；建议按 P0/P1 分批落地。

P0（高频 + 高破坏）

1. 输入：Backspace=DEL/BS 都应删除 1 字符（不删词）
2. 输入：Ctrl+C（中断）在请求中/空闲时都可预期（不崩溃、不污染 session）
3. 粘贴：bracketed paste 包含 `/help` 等前缀文本不得触发 REPL 命令分支
4. 粘贴：`/paste` 多行合并为一个 turn（含空行、含以 `/` 开头行）
5. 竞争：streaming 输出期间 typeahead（按回车/不按回车）不得丢行/合并 turn
6. Unicode：中文输入可正确落盘（Windows code page / UTF-8 两种）
7. 退出恢复：REPL 退出后终端仍可正常输入/回显（无 raw 残留、无光标消失）

P1（常见 + 难定位）

8. 流控：IXON 场景（Ctrl+S/Ctrl+Q）下不要误判“卡死”
9. 输出：`\r` 覆盖型输出不应破坏 prompt 逻辑（至少功能不丢输入）
10. resize：窗口尺寸变化时（ConPTY/pty）不会崩溃/卡死（若 TUI 才需要深入）
11. special keys：方向键/Home/End/Delete 不应把控制序列落为普通文本
12. alternate buffer：程序不应意外进入 alternate buffer（或应正确恢复）
13. 安全：输出中包含 OSC/CSI 控制序列时的默认防护策略（opt-in）

Sources

[1] Microsoft Learn — GetConsoleMode (authoritative; Windows console input mode flags).

<https://learn.microsoft.com/en-us/windows/console/getconsolemode>

[2] Microsoft Learn — SetConsoleMode (authoritative; input/output mode flags incl. wrap/newline behavior). <https://learn.microsoft.com/en-us/windows/console/setconsolemode>

[3] Debian Manpages — termios(3) (authoritative; ICANON/ECHO/IXON/WERASE etc).

<https://manpages.debian.org/bullseye/manpages-dev/termios.3.en.html>

- [4] XTerm Control Sequences (authoritative; bracketed paste, alternate screen, cursor/CSI/OSC). <https://www.invisible-island.net/xterm/ctlseqs/ctlseqs.html>
- [5] GNU Bash Reference Manual — Readline Init File Syntax (authoritative; enable-bracketed-paste semantics). https://www.gnu.org/s/bash/manual/html_node/Readline-Init-File-Syntax.html
- [6] XTerm Control Sequences — OSC “Manipulate Selection Data” (authoritative; selection/clipboard manipulation via OSC Ps=52). <https://www.invisible-island.net/xterm/ctlseqs/ctlseqs.html>
- [7] wcwidth documentation — Specification (high credibility; printable width rules, combining marks, East Asian width). <https://wcwidth.readthedocs.io/en/latest/specs.html>
- [8] wcwidth PyPI — ambiguous width behavior (high credibility; ambiguous_width parameter explains CJK preference differences). <https://pypi.org/project/wcwidth/>
- [9] Microsoft Learn — GetConsoleCP / GetConsoleOutputCP (authoritative; Windows console code pages). <https://learn.microsoft.com/en-us/windows/console/getconsolecp> ; <https://learn.microsoft.com/en-us/windows/console/getconsoleoutputcp>
- [10] The Register — ANSI escape sequences abuse / OSC52 discussion (secondary; security overview and motivation). https://www.theregister.com/2023/08/09/ansi_escape_sequence_risks/
- [11] XTerm FAQ — bracketed paste bypass & hardening notes (high credibility; points to real-world bypass patterns). <https://invisible-island.net/xterm/xterm-paste64.html>
- [12] Python bug tracker — readline 8.1 enables bracketed paste mode by default (high credibility; ecosystem regression example). <https://bugs.python.org/issue42819>
- [13] GNU Readline user manual (primary; bracketed paste option). <https://tiswww.case.edu/php/chet/readline/rluserman.html>
- [14] Microsoft Learn — Console Virtual Terminal Sequences (authoritative; VT input sequences incl. Backspace=0x7f). <https://learn.microsoft.com/en-us/windows/console/console-virtual-terminal-sequences>
- [15] Microsoft Learn — Creating a Pseudoconsole session (authoritative; ConPTY threading/drain/deadlock notes). <https://learn.microsoft.com/en-us/windows/console/creating-a-pseudoconsole-session>
- [16] Microsoft Learn — ClosePseudoConsole (authoritative; deadlock/drain warning; Win11 24H2 behavior change). <https://learn.microsoft.com/en-us/windows/console/closepseudoconsole>
- [17] Microsoft Learn — ReleasePseudoConsole (authoritative; ConPTY lifetime ownership guidance). <https://learn.microsoft.com/en-us/windows/console/releasepseudoconsole>
- [18] Microsoft Learn — SetConsoleCtrlHandler (authoritative; control handlers + Ctrl+C behavior notes). <https://learn.microsoft.com/en-us/windows/console/setconsolectrlhandler>
- [19] Microsoft Learn — GenerateConsoleCtrlEvent (authoritative; sending Ctrl+C/Ctrl+Break to console process groups). <https://learn.microsoft.com/en-us/windows/console/generateconsolectrlevent>
- [20] tmux Wiki — FAQ (high credibility; escape-time explanation and practical tradeoffs). <https://github.com/tmux/tmux/wiki/FAQ>
- [21] ncurses — terminfo(5) (authoritative; key_backspace/kbs and other key capability mappings).

<https://invisible-island.net/ncurses/man/terminfo.5.html>

[22] Debian Manpages — has_key(3ncurses) (high credibility; notes ESC key delay; key mapping caveats). https://manpages.debian.org/jessie/ncurses-doc/has_key.3ncurses.en.html

[23] GNU Coreutils Manual — stty invocation (high credibility; sane/raw/cooked combo settings). https://www.gnu.org/software/coreutils/manual/html_node/stty-invocation.html

[24] NO_COLOR (high credibility; informal standard for disabling ANSI color output; updated 2026-02-09). <https://no-color.org/>

[25] GNU gettext libtextstyle — The NO_COLOR variable (high credibility; adoption semantics, override notes). https://www.gnu.org/software/gettext/libtextstyle/manual/html_node/The-NO_005fCOLOR-variable.html

[26] Trojan Source (high credibility; paper landing page with arXiv link). <https://trojansource.codes/>

[27] NVD — CVE-2021-42574 (authoritative; Trojan Source / Bidi control characters). <https://nvd.nist.gov/vuln/detail/CVE-2021-42574>

[28] RustSec — RUSTSEC-2025-0055 / CVE-2025-58160 (high credibility; ANSI escape injection via logging). <https://rustsec.org/advisories/RUSTSEC-2025-0055>

[29] GitHub Advisory — CVE-2025-55193 (high credibility; ANSI escape injection via logging). <https://github.com/advisories/GHSA-76r7-hhxj-r776>

[30] microsoft/terminal — IME issue #14349 (high-signal engineering source; IME edge-case hangs/repeats). <https://github.com/microsoft/terminal/issues/14349>

Gaps and Further Research

- 建立“终端兼容矩阵”：cmd.exe / Windows Terminal / VS Code Terminal / ssh / tmux / WSL 的关键差异与最小覆盖集。
- 若要做更完备的 Unicode 支持：按 grapheme cluster 回删/对齐（可能需要引入额外依赖），并把它作为可选增强。
- 将 `conpty-expect` 扩展为通用框架：记录原始输入字节、可选屏幕快照、可复现 stress（种子/次数/超时配置）。
- 把“输出消毒/安全开关”设计成一致的 CLI contract：`NO_COLOR`（禁色）是社区共识之一，但“禁 OSC/禁 hyperlink/禁 title 改写”等更细粒度开关也值得规范化。[24][25]